

Curso de Especialización Ciberseguridad en las TIC

Práctica 1. Autenticación de doble factor por SSH con Google Authenticator

Objetivos

- Identificar correctamente el tipo de credencial utilizado en el sistema 2FA.
- Definir los mecanismos de autenticación en base a múltiples factores utilizados.
- Definir los protocolos y políticas de autenticación basados en tokens OTP utilizados.
- Poner en producción un sistema 2FA en un equipo Ubuntu.
- Poner en producción un sistema 2FA en un sistema Windows.

Resumen

Autora: Marina del Pilar López Oliva

Noviembre 2025

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	28/11/2025	Realización de la práctica.

Índice de Contenidos

1. Introducción.....	4
2. Apartado 1.....	4
3. Apartado 2.....	4
4. Apartado 3.....	5
5. Apartado 4.....	5
6. Apartado 5.....	5
7. Webgrafía.....	5

1. Introducción

En esta práctica vamos a configurar sistemas de control de acceso y autenticación de personas preservando la confidencialidad y privacidad de los datos. Utilizamos un sistema de autenticación de doble factor mediante Google Authenticator en el acceso ssh a un sistema GNU/Linux y Windows.

El documento se dividirá en cinco apartados. Los tres primeros apartados serán para identificar correctamente el tipo de credencial utilizado en el sistema 2FA, definir mecanismos de autenticación en base a múltiples factores utilizados en la práctica, y definir protocolos y políticas de autenticación basados en tokens OTP utilizados en esta práctica.

Por último, se realizarán dos apartados finales. Uno que contendrá el proceso realizado para la puesta en producción segura de un sistema 2FA en un equipo Ubuntu. Mientras que el otro será lo mismo pero en un sistema Windows.

2. Apartado 1

En el sistema de autenticación de doble factor que vamos a implementar en el apartado 4 y 5, se utilizan los siguientes tipos de credenciales:

- Credenciales basadas en conocimiento: la contraseña del usuario. Verifica la identidad mediante información memorizada por el usuario.
- Credenciales basadas en propiedad: dispositivo móvil con Google Authenticator configurado. Demuestra la posesión de un dispositivo físico específico y previamente configurado con el usuario.

Con estas credenciales protegemos contra amenazas y mejoramos la seguridad. Ya que aunque el atacante obtenga la contraseña no puede acceder sin el token. Además, el TOTP es inútil si se intercepta, pues caduca rápidamente y es muy difícil de interceptar. Y como la temporalidad del código es mínima, la ventana para atacar por fuerza bruta se limita considerablemente.

Con esto también multiplicamos las barreras que un atacante debe superar para poder acceder al usuario. Mitigamos el riesgo de impacto por las contraseñas débiles. Y además, proporcionamos una evidencia tangible del método de autenticación utilizado.

3. Apartado 2.

En el primer factor de verificación, hemos utilizado basados en el conocimiento. Algo que se sabe, cómo la contraseña del usuario en Ubuntu. Verificando, así inicialmente la identidad mediante información memorizada. Después, como segundo factor, hemos implementado algo que el usuario posee. Como un TOTP en un dispositivo con Google Authenticator. Con esto hemos verificado al usuario mediante posesión de un token.

Entonces, el factor 1 ha sido el conocimiento de la contraseña del usuario. Y el factor 2 la propiedad de una aplicación de autenticación, en este caso, un dispositivo con Google Authenticator.

El protocolo utilizado ha sido de desafío-respuesta:

- Factor 1:
 - Desafío: se le pide al usuario la contraseña.
 - Respuesta: el usuario pone la contraseña correcta para acceder.
 - Verificación: el servidor valida con la contraseña del usuario.
- Factor 2:
 - Desafío: se solicita el código de verificación tras la contraseña.
 - Respuesta: el usuario introduce el código temporal de la app.
 - Verificación: el servidor valida con la clave secreta compartida.

Se ha integrado el mecanismo PAM. Hemos modificado su configuración para que se pidiera tanto la contraseña como el código OTP de Google Authenticator. Esto combina factores distintos para el acceso mediante SSH.

Esta implementación sigue el esquema de múltiples factores. Con esto mejoramos la seguridad mediante la combinación de credenciales de conocimiento y propiedad.

Además, hemos implementado que nos pida la clave pública para SSH. Lo que también es considerado como algo que sabemos. Por lo que para acceder al usuario mediante ssh, debemos tener la clave pública del usuario, saber la contraseña y tener el dispositivo móvil con el Google Authenticator configurado. Esta combinación es una gran implementación de seguridad.

4. Apartado 3

Hemos implementado esta práctica TOTP mediante Google Authenticator. Con este se generan código de 6 dígitos válidos por 30 segundos. Gracias a esto hemos combinado el factor de conocimiento mediante una contraseña del usuario, y el factor de propiedad mediante un dispositivo móvil con Google Authenticator.

Se pueden encontrar las distintas vulnerabilidades:

- Ataques de relay: reutilización de credenciales interceptadas. Para mitigar esto se utilizan códigos de un solo uso no reutilizables.
- Fuerza bruta: 30 segundos igual es un tiempo demasiado largo, si queremos menos probabilidades, podríamos reducir el tiempo.
- Phishing: las credenciales serían inútiles si se nos intercepta.
- Robo de contraseñas: requiere un segundo factor de propiedad, como el dispositivo móvil.

Las políticas de seguridad que utilizamos son:

- Autenticación multifactor obligatoria para conexiones mediante ssh.
- Almacenamiento seguro de las claves OTP.
- Rotación automática cada 30 segundos de las claves OTP.
- Verificación en dos pasos secuencial: primero se pide contraseña y luego el código.

Con esto hemos llegado a fortificar:

- Defensa en profundidad: combinando claves basadas en conocimiento y basadas en propiedad.
- Protección de accesos remotos: mitigando los riesgos en las conexiones mediante ssh.
- Reducción de la superficie de ataque: invalidando las credenciales estáticas, pues los tokens OTP van rotando cada 30 segundos.
- Reducción ante amenazas.

5. Apartado 4

Lo primero que hemos hecho es instalar el paquete necesario para la autenticación de doble factor “libpam-google-authenticator”.

Después hemos ejecutado el asistente de configuración con “google-authenticator”. Aquí, le hemos permitido la autenticación basada en tiempo (TOTP), generamos el QR y la clave secreta y configuramos las opciones de

seguridad recomendada. Cuando sale el código QR nos conectamos a la aplicación de Google Authenticator con el QR.

A continuación, configuramos el archivo de PAM llamado sshd, añadiendo dos líneas para las reglas de autenticación.

```
auth requisite pam_unix.so nullok_secure
auth requisite pam_google_authenticator.so_
```

Figura 1. Archivo sshd del PAM modificada con las reglas de autenticación.

Después, editamos el archivo sshd_config, para permitir la autenticación mediante la aplicación. Para esto, simplemente añadimos “KbdInteractiveAuthentication yes”. Si queremos especificar que utilizar, lo haremos mediante “AuthenticationMethods”. En este caso, pediremos la clave pública y la clave de OTP de la aplicación.

```
AuthenticationMethods publickey,keyboard-interactive
```

Figura 2. Configuración ssh habilitando la autenticación interactiva.

Reiniciamos el servicio para que se activen los cambios.

Cuando quise comprobar si funciona correctamente esta configuración, me vi en la tesitura de necesitar una conexión forzada para el uso de clave. No entiendo a que se debió este error, sé que tiene que ver con la línea de “AuthenticationMethods”. Por ello, al ejecutar el comando utilicé “-i”, forzando el uso de clave específica para conectarse mediante ssh al usuario. Esto no me evitó demostrar la comprobación correcta de la configuración de la aplicación OTP.

Hemos podido comprobar que cuando intentamos entrar en un usuario, nos pide la contraseña y el código de verificación del Google Authenticator.

```
(marina@KaliMox)-[~]
$ ssh -i .ssh/iescamp marina@192.168.100.189
Enter passphrase for key '.ssh/iescamp':
(marina@192.168.100.189) Password:
(marina@192.168.100.189) Verification code:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-87-generic x86_64)
```

Figura 3. Secuencia de autenticación por ssh mostrando solicitud de clave pública, contraseña y código OTP de Google Authenticator.

A continuación, se nos pide que creamos una expresión condicional a aplicar para el contexto de mi proyecto. En mi caso, voy a crear un grupo para el departamento de informática. Y en él, añadiré un usuario que será un supuesto informático llamado Juan. Para ello, utilizaré el “sudo groupadd” y el “sudo adduser”. Posteriormente, añadiré el usuario al grupo mediante “sudo usermod -aG”.

```
marina@iescamp:~$ sudo groupadd dep_informatica
marina@iescamp:~$ sudo adduser marina
fatal: The user `marina' already exists.
marina@iescamp:~$ sudo adduser juan
info: Adding user `juan' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `juan' (1002) ...
info: Adding new user `juan' (1002) with group `juan (1002)' ...
info: Creating home directory `/home/juan' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for juan
Enter the new value, or press ENTER for the default
    Full Name []: Juan Chaves
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
info: Adding new user `juan' to supplemental / extra groups `users' ...
info: Adding user `juan' to group `users' ...
marina@iescamp:~$ sudo usermod -aG dep_informatica juan
```

Figura 4. Creación del grupo y usuario. Unión del usuario al grupo.

Con el grupo y usuario creado, vamos a crear una excepción. Con estas líneas en la configuración del PAM haremos que los usuarios del departamento de informática no requieran de una clave de la aplicación. Es decir, que tengan acceso rápido sin 2FA para, por ejemplo, tareas de mantenimiento crítico.

```
auth [success=done default=ignore] pam_succeed_if.so user ingroup dep_informatica
```

Figura 5. Adición de la excepción del grupo `dep_informatica` para 2FA.

Cuando queremos acceder con este usuario, perteneciente al grupo que hemos excepcionado, no nos pedirá la clave del Google Authenticator.

```
$ ssh -i .ssh/iescamp juan@192.168.100.189
(juan@192.168.100.189) Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-87-generic x86_64)
```

Figura 6. Conexión ssh con usuario del grupo que no requiere código OTP.

Por último, se nos plantean dos preguntas. Vamos a contestar y comprobar cada una de ellas para ver el correcto funcionamiento de toda la configuración realizada.

Si se inicia mediante el usuario del grupo `dep_informatica`, y se cambia a otro que tenga activo Google Authenticator, ¿nos solicitará código?

La respuesta es no. En nuestro ejemplo, hemos utilizado el “su” para conectarnos desde Juan al usuario de Marina. Como podemos ver, no se nos pide el código OTP.

```
juan@iescamp:~$ su marina
Password:
marina@iescamp:/home/juan$
```

Figura 7. Prueba cambio de usuario con su.

Esto ocurre porque la autenticación 2FA la hemos configurado solo para el acceso vía ssh. No hemos configurado Google Autenticar para el cambio de usuario posterior dentro de la misma sesión. El comando su cambia de usuario dentro de la misma sesión terminal.

¿Cómo resolverías el problema?

Deberíamos configurar el PAM para que también pida el 2FA en caso de utilizar su.

Para ello, accedemos al archivo su dentro del directorio del PAM. Y añadimos la misma configuración que para el sshd.

```
GNU nano 7.2 /etc/pam.d/su
auth sufficient pam_rootok.so
auth required pam_unix.so nullok_secure
auth [success=done default=ignore] pam_succeed_if.so user ingroup dep_informatica
auth required pam_google_authenticator.so
```

Figura 8. Archivo su del PAM modificada con las reglas de autenticación.

Ahora ya no debería ocurrir este problema. Si accedemos al usuario de Juan y queremos acceder con su al de Marina, nos pedirá doble verificación. Nos pedirá la contraseña del usuario al que queremos acceder, y si no pertenece al departamento de informática, también nos pedirá el código configurado en Google Authenticator.

```
last login: Fri Nov 20 13:13:12
juan@iescamp:~$ su marina
Password:
Verification code:
marina@iescamp:/home/juan$
```

Figura 9. Solicitud del código de verificación de Google Authenticator con acceso mediante el comando su.

Con esto, ya tenemos una gran configuración de seguridad para acceder a los usuarios fuera del grupo dep_informatica.

6. Apartado 5

Para esta parte de la práctica, vamos a utilizar Rohos para la configuración del 2FA. Lo primero que hay que hacer es descargar Rohos en la página oficial, y seguir la instalación mediante el asistente de instalación.



Figura 10. Asistente de instalación de Rohos Logon Key.

Después, configuraremos el Rohos para que nos pida contraseña del usuario "Administrador". Nos dirigiremos a "Setup OTP Token" y lo configuraremos para Google Authenticator.



Figura 11. Página inicial de la aplicación Rohos.

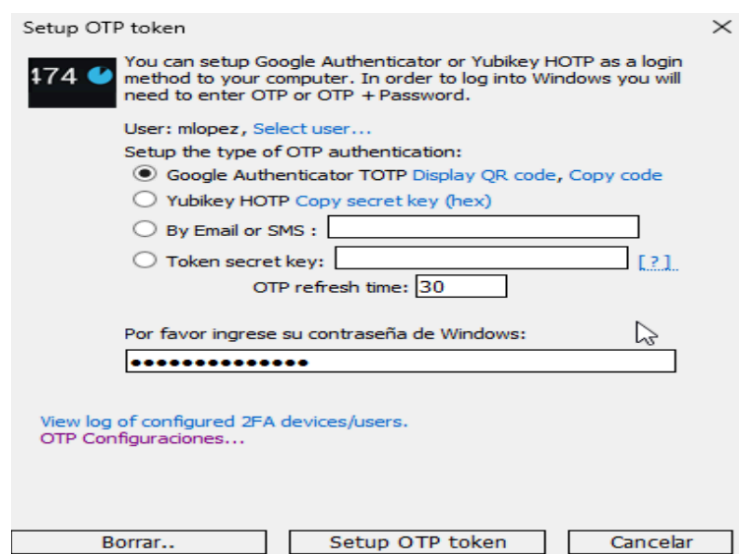


Figura 12. Configuración para el Google Authenticator.

Le daremos en “Display QR Code” para que nos muestre el código QR para sincronizar la aplicación. Este QR lo escaneamos con la aplicación y ya tendremos los tokens en el móvil.



Figura 13. QR a escanear para el Google Authenticator.

Cuando le demos a “Setup OTP Token” nos mostrará si todo está correcto, la siguiente pantalla.

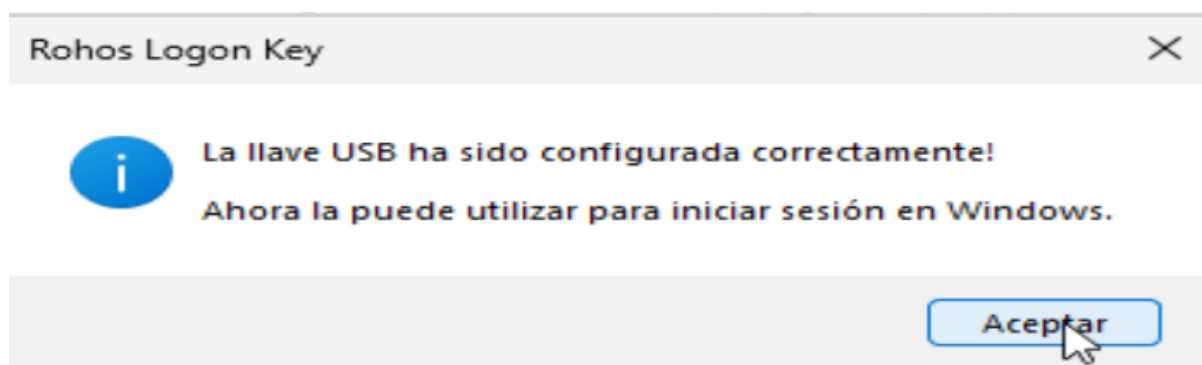


Figura 14. Correcta configuración.

Configuraremos que se nos requiera el token cuándo nos conectemos vía RDP.



Figura 15. Configuración para el requerimiento de la clave por conexión RDP.

Ahora ya está configurado, nos faltaría comprobar la funcionalidad de Rohos. Para ello, en Windows Server, tendremos que habilitar el escritorio remoto.

```
PS C:\WINDOWS\system32> Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server' -Name "fDenyTSConnections" -Value 0
```

Figura 16. Comando para habilitar las conexiones RDP.

Comprobamos la IP del Windows Server para poder conectarnos. En mi caso, es 192.168.1.157. Sabiendo esto, podemos volver a otro sistema e intentar la conexión mediante RDP. Para ello, utilizaremos en mi caso remmina. Crearemos una conexión por RDP al Windows Server.



Figura 17. Conexión para el RDP de Windows Server.

Una vez tenemos la conexión establecida, se nos pedirá que usemos el código de verificación configurado por Rohos.

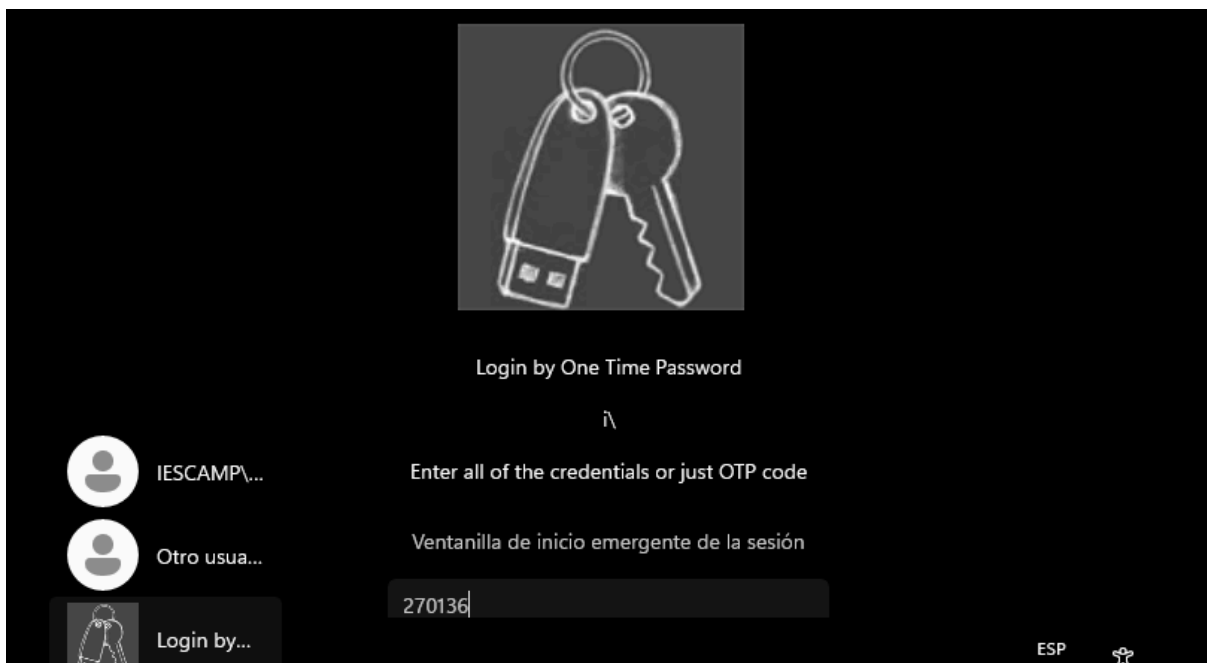


Figura 18. Verificación de la configuración de Rohos.

7. Webgrafía

Contenidos del capítulo 1 del UT3.

[[Página oficial del Rohos](#)]

[[Configuración de Google Authenticator en Rohos](#)]